

DOCUMENTUL DE PROIECTARE ARHITECTURALA

(Architectural Design Document)



Aplicatie de sah cu motor de inteligenta artificiala
si modul multiplayer online

Cuprins

Acest document descrie arhitectura sistemului ChessMate, modul de impartire pe subsisteme, distributia componentelor pe platforme hardware, organizarea datelor persistente si fluxurile principale de control.

Cuprins.....	2
1. Introducere	3
1.1. Scopul sistemului	3
1.2. Definitii si acronime	3
1.3. Documente de referinta	4
2. Obiective de proiectare	4
3. Arhitectura propusa.....	5
3.1. Decompozitia in subsisteme.....	5
Stratul de prezentare	5
Stratul de aplicatie (server)	5
Stratul de domeniu (logica de joc).....	6
Stratul de persistenta	6
Interfetele dintre subsisteme.....	6
3.2. Distributia subsistemelor pe platforme hardware/software	6
Note de distributie.....	7
3.3. Managementul datelor persistente	8
Entitatile principale.....	8
Indecsi recomandati	9
Strategii de stocare	9
3.4. Controlul accesului utilizatorilor la sistem	9
Autentificare.....	9
Roluri si permisiuni	9
Verificarea permisiunilor.....	10
Protectie impotriva atacurilor comune.....	10
3.5. Fluxul global al controlului	10
3.6. Tratarea conditiilor limita.....	11
Pornirea sistemului.....	12
Oprirea sistemului	12
Tratarea erorilor	12
Cazuri de utilizare administrative	12
Glosar de termeni.....	13

1. Introducere

1.1. Scopul sistemului

ChessMate este o aplicatie completa de sah care combina trei moduri de joc intr-o singura platforma: joc impotriva unui motor de inteligenta artificiala cu niveluri ajustabile de dificultate, joc local intre doi jucatori pe acelasi dispozitiv si joc online in timp real impotriva altor utilizatori inregistrati.

Documentul de fata descrie deciziile de proiectare arhitecturala care stau la baza sistemului: modul in care responsabilitatile sunt distribuite intre componente, cum sunt organizate datele, cum interactioneaza partea client cu partea server si cum se trateaza situatiile speciale (pornire, oprire, erori, cazuri administrative).

Audienta principala este formata din echipa de dezvoltare (developer backend, frontend, motor de joc), echipa de testare si responsabilul de implementare. Documentul serveste ca punct comun de referinta inaintea inceperii implementarii efective.

1.2. Definitii si acronime

Termen / acronim	Semnificatie
ADD	Architectural Design Document - documentul de fata
SRS	Software Requirements Specification - documentul de specificare a cerintelor
IA / AI	Inteligenta artificiala - motorul care joaca impotriva utilizatorului
JWT	JSON Web Token - format de token folosit pentru autentificare
WSS	WebSocket Secure - protocol de comunicare bidirectionala criptata
REST	Representational State Transfer - stil arhitectural pentru API-uri HTTP
FEN	Forsyth-Edwards Notation - notatie textuala standard pentru o pozitie de sah
ELO	Sistem numeric de evaluare a fortei jucatorilor
PvP	Player versus Player - joc intre doi utilizatori umani
TLS	Transport Layer Security - protocol de criptare a traficului de retea
ORM	Object-Relational Mapping - punte intre cod si baza de date relationala

1.3. Documente de referinta

- Documentul de specificare a cerintelor (SRS) - ChessMate, versiunea 1.0
- Standard FIDE - regulile oficiale ale jocului de sah
- RFC 6455 - The WebSocket Protocol
- RFC 7519 - JSON Web Token (JWT)
- PostgreSQL 15 - documentatia oficiala pentru baza de date

2. Obiective de proiectare

Arhitectura sistemului este construita pentru a satisface in egala masura cerintele functionale si calitative din SRS. Cele mai importante obiective care au influentat deciziile de proiectare sunt urmatoarele:

- **Separare clara intre logica de joc si infrastructura.** Motorul de sah si motorul IA sunt complet independente de partea de retea, baza de date sau interfata. Aceasta separare permite testarea izolata a regulilor de joc si reutilizarea engine-ului in modul offline.
- **Capacitate de scalare orizontala.** Serverul de aplicatie nu retine stare in memoria proprie pentru sesiunile HTTP; partidele active sunt salvate in baza de date dupa fiecare mutare, astfel incat doua replici ale serverului sa poata prelua aceeasi partida.
- **Latenta scazuta in modul multiplayer.** Comunicatia in timp real foloseste WebSocket pentru a evita overhead-ul polling-ului, iar mutarile se trimit ca mesaje compacte (notatie algebrica).
- **Securitate prin proiectare.** Toate canalele externe sunt criptate (TLS), parolele sunt stocate cu hashing puternic (bcrypt), autentificarea foloseste tokenuri JWT cu expirare, iar accesul la baza de date este restrictionat la reseaua interna a serverului.
- **Disponibilitate si robustete.** Sistemul tolereaza deconectarea temporara a unui client fara a pierde partida; reconectarea readuce starea actualizata din server.
- **Mentenabilitate.** Codul este organizat pe straturi (prezentare, aplicatie, domeniu, persistenta), cu interfete clare intre ele. Adaugarea unui mod nou de joc sau a unei rute noi de API nu necesita modificari in motorul de sah.
- **Portabilitate la nivel de client.** Interfata web ruleaza pe orice browser modern, fara instalare. Aplicatia desktop optionala foloseste acelasi cod sursa al motorului si interfetei.

3. Arhitectura propusa

3.1. Decompozitia in subsisteme

Sistemul este organizat pe patru straturi logice, fiecare cu responsabilitati bine delimitate. Aceasta organizare urmeaza modelul clasic in straturi (layered architecture), in care fiecare nivel comunica numai cu nivelul imediat inferior prin interfete clar definite.

ChessMate - Diagrama de componente

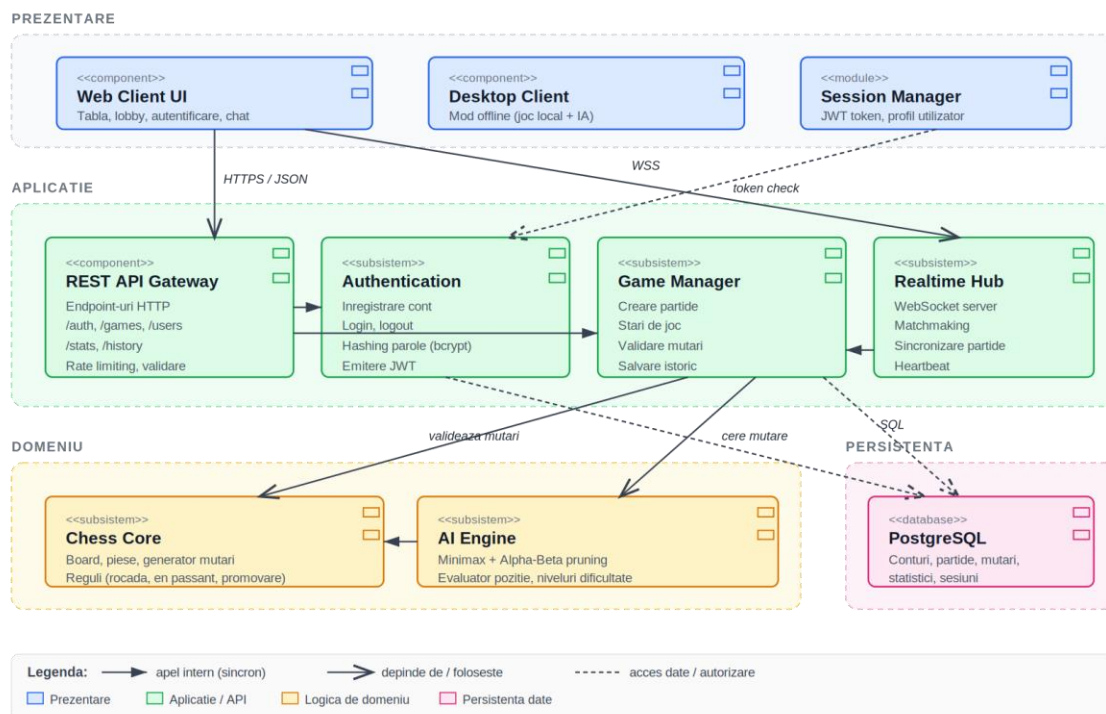


Figura 1. Diagrama de componente - decompozitia in subsisteme

Stratul de prezentare

Stratul de prezentare contine tot ce ruleaza pe dispozitivul utilizatorului. Componenta principala este Web Client UI, o aplicatie web monopagina care randeaza tabla de sah, gestioneaza inputul utilizatorului si afiseaza ecranele de meniu, lobby, statistici si chat. Optional, exista si un Desktop Client pentru rularea offline (joc local sau impotriva IA, fara conectare la server). Session Manager este modulul care pastreaza tokenul JWT in memoria browserului si il ataseaza la fiecare cerere catre server.

Stratul de aplicatie (server)

La nivelul serverului exista patru subsisteme principale. REST API Gateway expune punctele de acces HTTP pentru autentificare, gestiunea conturilor, statistici si istoric. Authentication se ocupa exclusiv de inregistrare, login, logout, hashing parole (bcrypt) si emiterea tokenurilor JWT. Game Manager coordoneaza ciclul de viata al unei partide: creare, validare mutari (deleaga catre Chess Core), salvare

istoric, finalizare. Realtime Hub mentine conexiunile WebSocket active, realizeaza matchmaking-ul (asocierea a doi jucatori care cauta partida) si difuzeaza mutarile intre jucatori in timp real.

Stratul de domeniu (logica de joc)

Aici se afla cei doi piloni ai jocului. Chess Core este implementarea regulilor de sah: reprezentarea tablei, generatorul de mutari legale, regulile speciale (roca, en passant, promovare), detectia situatiilor de sah, sah-mat si pat. AI Engine este motorul de inteligenta artificiala bazat pe algoritmul Minimax cu optimizare Alpha-Beta pruning, cu trei niveluri de dificultate (adancime de cautare 2, 4 si 6) si o functie de evaluare bazata pe valoarea materiala si factori pozitionali.

Stratul de persistenta

Toate datele care trebuie sa supravietuiasca opririi serverului sunt stocate intr-o baza de date PostgreSQL: conturi de utilizator, partide, mutari, sesiuni active si statistici. Detalii despre schema sunt prezentate in sectiunea 3.3.

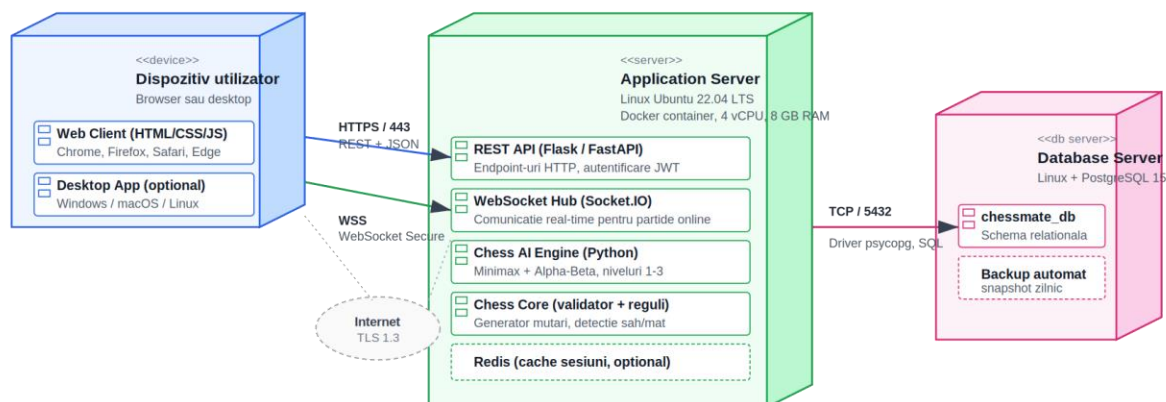
Interfetele dintre subsisteme

Stratul de prezentare comunica cu serverul prin doua canale: HTTPS pentru cereri ocazionale (login, citire profil, istoric) si WSS pentru fluxul continuu de mutari in modul multiplayer. Intern, in cadrul serverului, subsistemele comunica prin apeluri sincrone in cadrul aceluiasi proces. Game Manager este punctul central care orchestreaza interactiunea cu Chess Core (validare mutari) si AI Engine (calcul mutare adversar). Toate subsistemele cu nevoie de persistenta acceseaza baza de date printr-un strat ORM comun.

3.2. Distributia subsistemelor pe platforme hardware/software

Sistemul este distribuit pe trei tipuri de noduri fizice: dispozitivul utilizatorului (variabil ca tip), serverul de aplicatie si serverul de baza de date. Diagrama de distributie de mai jos arata cum sunt mapate componentele software pe aceste noduri.

ChessMate - Diagrama de distributie



Note de distributie

Modul offline: aplicatia desktop ruleaza Chess Core si AI Engine local; nu necesita conexiune la server.

Modul multiplayer: clientul mentine doua canale - HTTPS pentru request-uri ocazionale (login, statistici) si WSS pentru fluxul de mutari.

Scalare: Application Server poate fi replicat orizontal in spatele unui load balancer; Realtime Hub foloseste sticky sessions sau Redis pub/sub.

Securitate: tot traficul extern este criptat (TLS 1.3); accesul la baza de date este permis doar din retea internă a serverului.

Backup: snapshot zilnic al bazei de date, retentie 7 zile.

Figura 2. Diagrama de distributie - mapare componente pe hardware

Dispozitivul utilizatorului poate fi orice calculator modern cu un browser actualizat (Chrome 100+, Firefox 100+, Edge 100+, Safari 16+) sau, optional, un PC pe care a fost instalata aplicatia desktop. In modul offline, dispozitivul utilizatorului contine si Chess Core si AI Engine, care ruleaza local fara conexiune la server.

Application Server ruleaza pe Linux Ubuntu 22.04 LTS intr-un container Docker, cu 4 vCPU si 8 GB RAM. Pe acelasi nod gazduieste REST API, WebSocket Hub, Chess AI Engine, Chess Core si, optional, o instanta Redis pentru cache de sesiuni atunci cand serverul este replicat.

Database Server ruleaza separat pe un nod dedicat cu PostgreSQL 15 si este accesibil doar din retea internă. Conexiunea de la Application Server la baza de date se face pe portul 5432 cu driverul psycopg.

Note de distributie

- **Modul offline.** Aplicatia desktop poate functiona complet izolat - Chess Core si AI Engine sunt impachetate impreuna cu interfata, fara apeluri catre server.
- **Scalare.** Application Server poate fi multiplicat orizontal in spatele unui load balancer. Pentru a mentine consistenta sesiunilor WebSocket se folosesc fie sticky sessions, fie un mecanism de pub/sub bazat pe Redis.
- **Securitate.** Tot traficul intre client si server este criptat cu TLS 1.3. Baza de date nu este expusa public; firewallul permite acces doar din retea privata.
- **Backup.** Snapshot zilnic al bazei de date, cu retentie de 7 zile pe un volum separat.

3.3. Managementul datelor persistente

Toate datele care trebuie pastrate intre sesiuni sunt stocate intr-o baza de date relationala PostgreSQL. Alegerea PostgreSQL este motivata de suportul puternic pentru tipuri complexe (UUID, INET, ENUM), respectarea ACID, performanta buna pe interogari analitice (statistici, leaderboard) si fiabilitatea dovedita in productie.

In modul de dezvoltare se poate folosi SQLite pentru rapiditate si simplitate, prin acelasi strat ORM, astfel incat trecerea la PostgreSQL in productie sa fie transparenta.

Schema conceptuala a bazei de date este prezentata in figura urmatoare. Sunt evidentiata entitatile principale, attributele lor si relatiile dintre ele cu cardinalitatile corespunzatoare.

ChessMate - Schema conceptuala a bazei de date

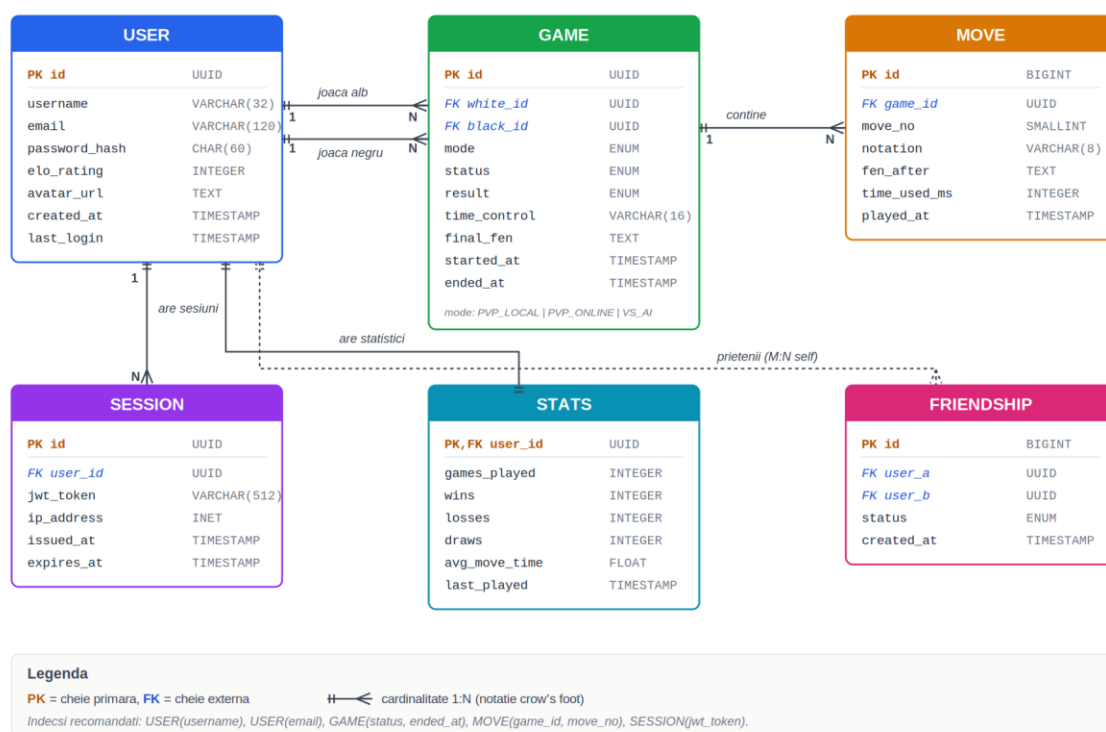


Figura 3. Schema conceptuala a bazei de date

Entitatile principale

- **USER**. Reprezinta un cont de utilizator inregistrat. Stocheaza credentialele (username, email, hash-ul parolei), avatarul, ratingul ELO curent si timpul ultimei autentificari.
- **GAME**. O partida de sah, indiferent de mod. Retine cei doi jucatori (`white_id`, `black_id`), modul (PVP_LOCAL, PVP_ONLINE, VS_AI), starea (in curs, finalizat, abandonat), rezultatul, controlul de timp, pozitia finala in notatie FEN si timestampurile de inceput / sfarsit.

- **MOVE.** O mutare individuala dintr-o partida. Pastreaza ordinea (move_no), notatia algebrica, pozitia rezultata (FEN) si timpul folosit. Permite rejucarea (replay) oricarei partide salvate.
- **SESSION.** O sesiune de autentificare activa - tokenul JWT emis, IP-ul si momentul expirarii. Permite invalidarea explicita a sesiunilor (logout pe toate dispozitivele).
- **STATS.** Statistici agregate per utilizator (numar de partide, victorii, infrangeri, remize, timp mediu pe mutare). Sunt actualizate la finalul fiecarei partide.
- **FRIENDSHIP.** Tabel de jonctiune pentru relatia M:N de prietenie intre utilizatori, cu un camp de status (pending, accepted, blocked).

Indecsi recomandati

Pentru performanta interogarilor frecvente se definesc indecsi pe: USER(username) si USER(email) pentru autentificare, GAME(status, ended_at) pentru lista partidelor active, MOVE(game_id, move_no) pentru reconstruirea ordonata a unei partide si SESSION(jwt_token) pentru validarea rapida la fiecare cerere.

Strategii de stocare

- Tranzactii: orice mutare se salveaza intr-o tranzactie unica impreuna cu actualizarea starii partidei (atomicitate).
- Soft delete: conturile sterse se marcheaza cu un flag deleted_at, nu se sterg fizic, pentru a pastra integritatea istoricului.
- Backup: snapshot zilnic complet, plus arhivare WAL pentru point-in-time recovery in cele 7 zile precedente.

3.4. Controlul accesului utilizatorilor la sistem

Accesul la sistem este controlat prin trei mecanisme complementare: autentificare bazata pe JWT, rolurile utilizatorilor si verificarea permisiunilor la nivel de subsistem.

Autentificare

Procesul de autentificare urmareste pasii standard pentru o aplicatie web moderna. La inregistrare, parola este transformata cu bcrypt (10 runde, salt aleator) si stocata in baza de date sub forma hash-ului. La login, parola transmisa este comparata cu hash-ul stocat; in caz de potrivire, serverul emite un token JWT semnat cu o cheie secreta. Tokenul contine identificatorul utilizatorului, rolul si timpul de expirare (24 de ore implicit).

La fiecare cerere ulterioara, clientul ataseaza tokenul in antetul HTTP Authorization. Serverul valideaza semnatura, expirarea si existenta sesiunii in tabelul SESSION. Cererile fara token valid primesc raspuns HTTP 401.

Roluri si permisiuni

Rol	Permisuni
Vizitator (neautentificat)	Vizualizare pagina principala, inregistrare cont, joc local PvP fara salvare.
Utilizator inregistrat	Toate functiile de joc (PvP local, vs IA, multiplayer online), istoric partide proprii, statistici, profil, prietenii.
Administrator	Toate functiile de mai sus, plus gestiunea conturilor (suspendare, stergere), vizualizare statistici globale, moderare chat.

Verificarea permisiunilor

Fiecare endpoint din REST API si fiecare eveniment WebSocket trec printr-un middleware de autorizare care verifica daca rolul utilizatorului are dreptul la operatiunea ceruta. De exemplu, vizualizarea istoricului unei partide este permisa doar pentru participantii la acea partida sau pentru administratori. Operatiunile administrative sunt protejate suplimentar prin verificarea rolului 'admin' in tokenul JWT.

Protectie impotriva atacurilor comune

- **Brute force la login.** Limitare la 5 incercari esuate per IP per minut, urmata de un cooldown de 15 minute.
- **Injectie SQL.** Toate interogările folosesc parametri pregatiti prin ORM, nu concatenare de string-uri.
- **XSS.** Continutul controlat de utilizator (username, mesaje chat) este escapat la afisare.
- **CSRF.** Cererile state-changing necesita token JWT in antet, nu cookie - fluxul este imun la atacurile CSRF clasice.

3.5. Fluxul global al controlului

Diagrama de mai jos prezinta fluxul global de control al sistemului, de la deschiderea aplicatiei pana la finalul unei partide. Sunt evidentiuate trei zone de responsabilitate (swimlanes): utilizatorul, clientul (UI) si serverul impreuna cu motorul.

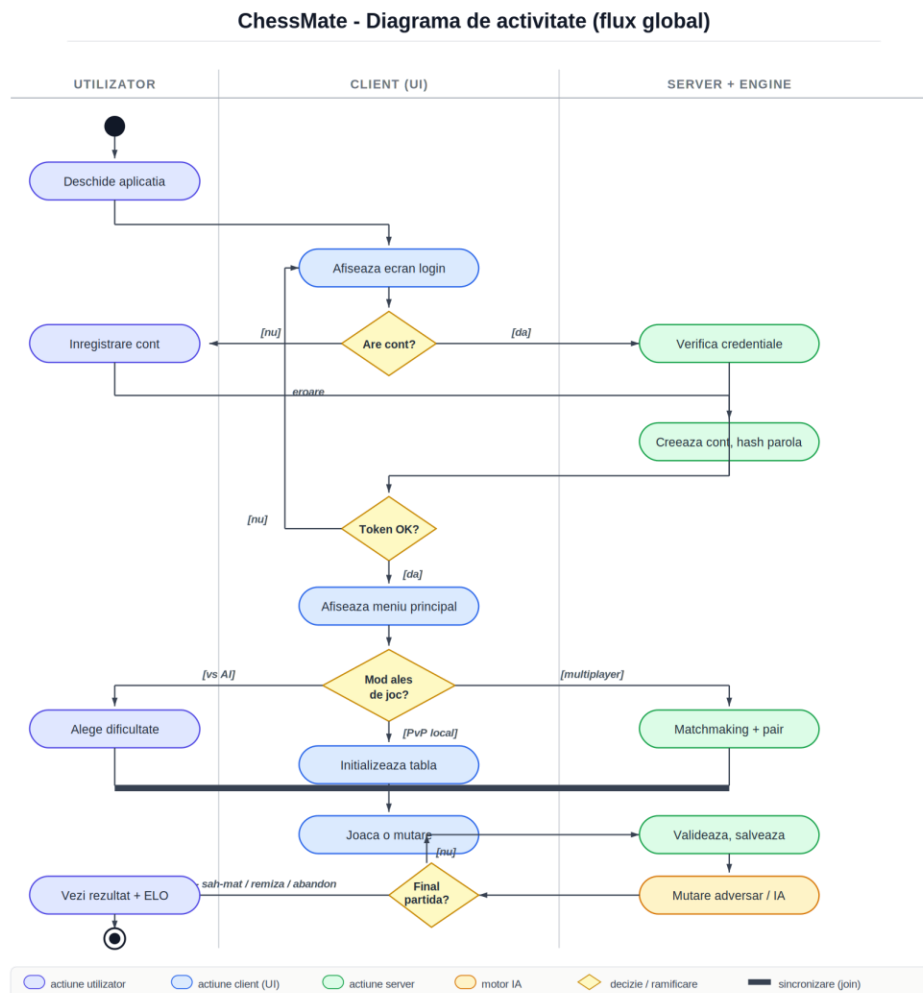


Figura 4. Diagrama de activitate - fluxul global de control

Fluxul incepe cu deschiderea aplicatiei. Daca utilizatorul nu are deja un cont, ramura de inregistrare il duce catre crearea unui cont nou (cu hashing al parolei pe server). Daca are cont, sunt verificate credentialele si se emite un token JWT. In ambele cazuri se ajunge la o decizie de validare a tokenului - in caz de esec, fluxul revine la ecranul de login cu un mesaj de eroare.

Dupa autentificare cu succes, utilizatorul vede meniul principal si alege modul de joc. Cele trei ramuri (vs IA, PvP local, multiplayer) converg printr-un nod de sincronizare in bucla principala de joc. In bucla, utilizatorul joaca o mutare, serverul o valideaza si o salveaza, apoi adversarul (IA sau jucator remote) executa mutarea proprie. Decizia de final de partida (sah-mat, remiza sau abandon) inchide bucla; in caz contrar, controlul revine la pasul de mutare pentru tura urmatoare. La final, utilizatorul vede rezultatul si ratingul ELO actualizat.

3.6. Tratarea conditiilor limita

Aceasta sectiune descrie comportamentul sistemului in situatii speciale: pornire si oprire ordonata, reactie la erori, pierdere de conexiune si cazurile de utilizare administrative.

Pornirea sistemului

La pornirea serverului se executa, in aceasta ordine:

1. Citire fisier de configurare (port, credentiale baza de date, cheie JWT, parametri IA).
2. Verificare conexiune cu baza de date si rulare migratiilor pendinte.
3. Pornire REST API pe portul HTTP configurat si WebSocket Hub pe portul de socket.
4. Curatare sesiuni expirate din tabelul SESSION.
5. Anuntare prin endpoint /health ca sistemul este pregatit (pentru load balancer).

Oprirea sistemului

La oprire (graceful shutdown) serverul refuza conexiuni noi, asteapta finalizarea cererilor HTTP in curs (cu un timeout de 30 secunde), notifica clientii WebSocket activi cu un mesaj de tip 'server_shutting_down', salveaza starea oricaror partide active in baza de date si inchide ordonat conexiunile la baza de date. Daca timpul de gratiere este depasit, conexiunile ramase sunt inchise fortat dar starea partidelor este deja persistata.

Tratarea erorilor

Situatie	Comportament
Pierdere conexiune client	Clientul detecteaza printr-un heartbeat la 30 secunde; la reconectare, cere starea curenta a partidei prin REST si reia fluxul WebSocket fara pierdere de mutari.
Cadere baza de date	Cererile in curs primesc HTTP 503; serverul reincearca conexiunea cu backoff exponential; partidele in curs se pun in pauza.
Mutare invalida	Serverul respinge mutarea cu un mesaj explicativ; clientul revine la starea anterioara fara a inchide partida.
Crash al motorului IA	Mutarea curenta este abandonata, partida se incheie ca remiza tehnica si se logheaza incidentul.
Token JWT expirat in timpul partidei	Clientul detecteaza HTTP 401 la urmatoarea cerere, solicita un token nou prin refresh, apoi reia operatiunea fara pierderea partidei.

Cazuri de utilizare administrative

- **Suspendare cont.** Administratorul poate suspenda un cont; toate sesiunile active sunt invalidate imediat (stergere din SESSION) si conturile suspendate nu se mai pot autentifica.

- **Stergere cont.** Stergere logica (soft delete); datele de profil sunt anonimizate (username inlocuit cu 'deleted_user_<id>'), istoricul partidelor se pastreaza pentru consistenta clasamentelor adversarilor.
- **Migrare schema.** Modificari ale schemei bazei de date sunt aplicate prin migratii versionate (Alembic / Flyway); serverul refuza pornirea daca exista migratii pendinte si configurarea cere validare stricta.
- **Backup si restore.** Snapshot complet zilnic + arhivare WAL pentru point-in-time recovery. Restore dintr-un snapshot necesita oprirea serverului, refacerea bazei de date din arhiva si repornirea cu acelasi fisier de configurare.
- **Monitorizare.** Endpoint /health pentru verificare status; metrice expuse pentru Prometheus (numar conexiuni active, latenta medie, rata erorilor); log-uri structurate JSON trimise centralizat.

Glosar de termeni

In completarea sectiunii 1.2, sunt definiti aici cativa termeni specifici domeniului care apar in document.

Termen	Definitie
Alpha-Beta pruning	Optimizare a algoritmului Minimax care elimina ramurile arborelui de cautare ce nu pot influenta decizia finala.
Bcrypt	Algoritm de hashing pentru parole, cu salt si cost configurabil, rezistent la atacuri brute force.
Endpoint	Adresa de retea (URL si metoda HTTP) prin care un client acceseaza o functionalitate a serverului.
Heartbeat	Mesaj periodic schimbat intre client si server pentru a verifica daca conexiunea este inca activa.
Matchmaking	Procesul prin care doi jucatori care cauta partida sunt asociati automat de catre server.
Middleware	Cod care intercepteaza cererile inainte de a ajunge la handler-ul propriu-zis (ex: autentificare, logare, rate limiting).
Migratie	Set de instructiuni versionate pentru a aduce schema bazei de date de la o versiune la urmatoarea.
Sticky session	Tehnica de load balancing care directioneaza toate cererile aceluasi client catre acelasi server.

Termen	Definitie
Swimlane	Banda verticala dintr-o diagrama de activitate care grupeaza pasii executati de acelasi actor sau component.